GUIDE 50 · GOVERN

Rotate an API key without downtime

The latest version is at anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=50-rotate-an-api-key-without-downtime. This copy was made on 4 November 2026.

Time: 10 minutes. **You'll end up with:** a partner's key on a new secret, with the old one working until they've switched, then stopped. Same key name, same grants, the same history.

Every command and response below was run against a real server on `http://127.0.0.1:5098`, with a PostgreSQL 16 `orders` table behind the saved query `orders_by_customer` (new in 0.22).

Why not a new key?

Before 0.22, "rotating" meant creating a second key, copying the grants across by hand, and revoking the first. The partner had an outage unless both of you switched at the same minute, the copied grants could differ, and run history and the audit log split across two names. Reissuing the secret avoids all three.

Step 1: The partner's key, working

A key that may run one query (guide 14):

```shell
curl -X POST http://127.0.0.1:5098/api/v1/api-keys -H "X-API-Key: $TOKEN" -H 'Content-Type:
application/json' \
  -d '{"name": "partner-acme", "connections": [], "queries": ["orders_by_customer"]}'
curl 'http://127.0.0.1:5098/q/orders_by_customer?customer=acme' -H "X-API-Key: $OLD_SECRET"
# [{"id":1,"total":120.5},{"id":2,"total":80.0}]
```

Step 2: Rotate the secret, keeping the old one for a day

```shell
curl -X POST http://127.0.0.1:5098/api/v1/api-keys/partner-acme/rotate -H "X-API-Key: $TOKEN" \
  -H 'Content-Type: application/json' -d '{"grace_minutes": 1440}'
```

```json
{"name": "partner-acme", "secret": "sk_quSL2_SHS6CWDGOQMyOW6XH8q0tbiA7mfRRILykmwRQ",
 "previous_expires_at": "2026-10-12 18:28:43", "previous_last_used_at": null,
 "queries": ["orders_by_customer"], ...}
```

The new `secret` is in this response only, sent with `Cache-Control: no-store`, as when the key was created. Give it to the partner. Until `previous_expires_at` **both** secrets work:

```shell
curl 'http://127.0.0.1:5098/q/orders_by_customer?customer=acme' -H "X-API-Key: $OLD_SECRET"   #
[{"id":1,...}]
curl 'http://127.0.0.1:5098/q/orders_by_customer?customer=acme' -H "X-API-Key: $NEW_SECRET"   #
[{"id":1,...}]
```

- `grace_minutes` is 60 if you leave it out, at most 10080 (a week).
- Both secrets are the same key: its `active`, expiry, `allowed_ips` and rate limit apply to each, and they share one rate-limit budget - the old secret is never a second allowance.
- Who may rotate: anyone who may change keys (`access.write` - owners and admins).

In the Console: **API keys** → **Edit** (the pencil) → **Secret**: choose how long to keep the old secret working, then **Rotate secret**. The new secret is shown once, and the key gets a *rotating* tag in the list.

Step 3: See whether they've switched

```shell
curl http://127.0.0.1:5098/api/v1/api-keys/partner-acme -H "X-API-Key: $TOKEN"
# "last_used_at": "2026-10-11 18:28:43", "previous_expires_at": "2026-10-12 18:28:43",
# "previous_last_used_at": "2026-10-11 18:28:43"
```

`previous_last_used_at` is when the **old** secret was last used (to the minute). Once it stops moving, they've switched. The Console's Edit drawer shows the same: *The old secret works until … · last used ….*

Step 4: End the old secret

Once they have switched, end it rather than waiting for the grace period to run out:

```shell
curl -X DELETE http://127.0.0.1:5098/api/v1/api-keys/partner-acme/previous -H "X-API-Key: $TOKEN"
# "previous_expires_at": null, "previous_last_used_at": null
curl 'http://127.0.0.1:5098/q/orders_by_customer?customer=acme' -H "X-API-Key: $OLD_SECRET"
# {"error":"Unauthorized","code":"unauthorized"}
```

In the Console, **End now**. Left alone, the old secret stops by itself at `previous_expires_at`.

One rotation at a time. Rotating again while an old secret still works is refused, so a second click can't cut off the secret the partner is still using:

```json
{"error": "The secret this key had before its last rotation works until 2026-10-12 18:28:43 - end that
first
 (DELETE .../previous), or wait for it", "code": "key_rotation_pending", "previous_expires_at": "2026-
10-12 18:28:43"}
```

If a secret leaked

Rotate with no grace period - the old secret stops at once:

```shell
curl -X POST http://127.0.0.1:5098/api/v1/api-keys/partner-acme/rotate -H "X-API-Key: $TOKEN" \
  -H 'Content-Type: application/json' -d '{"grace_minutes": 0}'
# "previous_expires_at": null
```

In the Console: **Keep the old secret working: not at all - it stops now**.

What's on record

The audit log, under the key's name, with who did it (`GET /api/v1/audit?target=partner-acme`) - never a secret:

```json
{"actor": "admin", "via": "break-glass", "action": "rotate_api_key", "changes": {"grace_minutes": 0,
"previous_expires_at": null}}
{"actor": "admin", "via": "break-glass", "action": "end_previous_secret", "changes": {"was_valid_until":
"2026-10-12 18:28:43"}}
{"actor": "admin", "via": "break-glass", "action": "rotate_api_key", "changes": {"grace_minutes": 1440,
"previous_expires_at": "2026-10-12 18:28:43"}}
```

Run history stays under `partner-acme` whichever secret made the call (`GET /api/v1/history?key=partner-acme`).

Good to know

- **Several instances:** every instance must run 0.22 or later before you rotate a key - an older one doesn't know the old secret, so calls reaching it with the old secret are refused.
- **Admin tokens** rotate differently: an administrator can hold several, so issue another and revoke the old one (guide 43).
- **The encryption key** for stored passwords rotates too: guide 22's Changing the key.

Next steps

- Give an external partner access to exactly one query.
- Give a key an expiry date - access that ends by itself.
- Read the audit log.